Spring MVC Security - Logout - Overview and Coding

In this lesson, we're going to explore how to implement Spring Security Logout. We'll add a new "Logout" button to our application's homepage. When clicked, this button will clear the user's session and redirect them back to the login page.

Let's take a look at our development process:

1. **Adding Logout Support to the Spring Security Configuration**: We'll include the `logout.permitAll` command to enable logout support. This exposes the default URL for logging out, `/logout`. Simple and straightforward.

File: DemoSecurityConfig.java

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

    http.authorizeHttpRequests(configurer ->
                    configurer
                            .anyRequest().authenticated()
            )
            .formLogin(form ->
                    form
                            .loginPage("/showMyLoginPage")
                            .loginProcessingUrl("/authenticateTheUser")
                            .permitAll()
            )
            .logout(logout -> logout.permitAll()
            );

    return http.build();
}
```

Add logout support for default URL /logout

2. **Adding a Logout Button**: This button will send data to the default logout URL, `/logout`, a feature provided by the Spring Security Filters. The GET method is disabled by default, so we'll use the POST method inside a form. When a logout is processed, Spring Security will invalidate the user's HTTP session, remove session cookies, and redirect them to the login page with a logout parameter appended.

```
<form action="#" th:action="@{/logout}" method="POST">

      <input type="submit" value="Logout" />

</form>
```

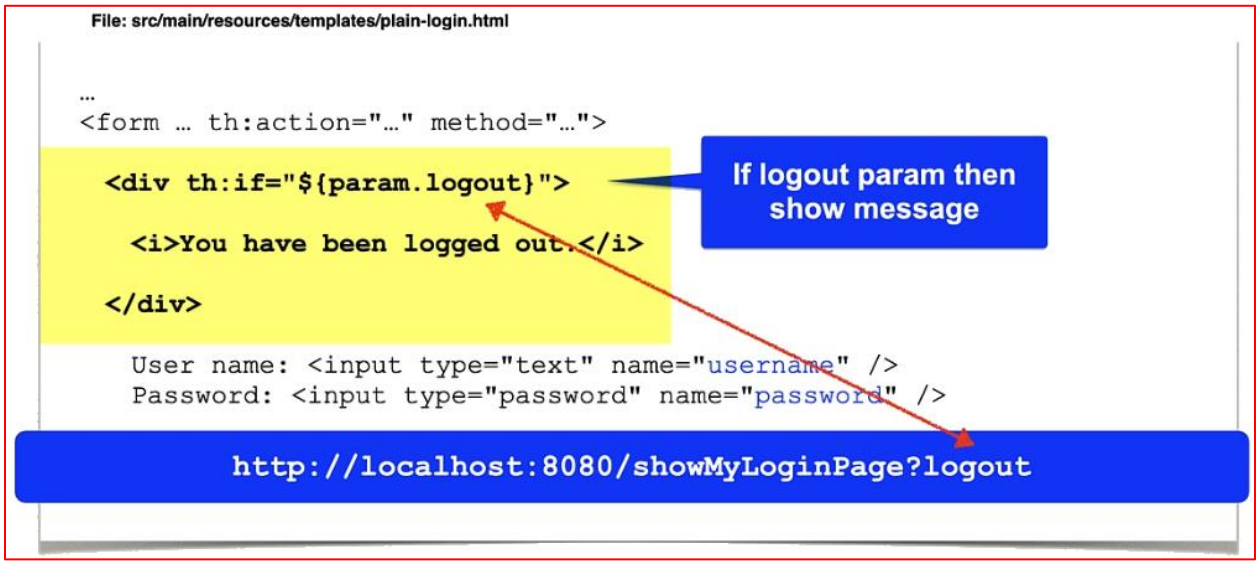
3. **Updating the Login Form to Display a "Logged Out" Message**: We'll modify the login form to check for the logout parameter and display a message if found, such as "Hey, you've been logged out."

Alright, let's get into the details:

Step One: In the Spring Security Configuration, the code snippet `.logout.permitAll` gives us logout support. This is part of Spring Security's helpful features.

Step Two: Adding the logout button requires sending data to the `/logout` URL. Here's our form action: `/logout method=POST`. We must post the data and place the input type or button inside this form. Note that by default, Spring Security will handle session invalidation, removal of cookies, and redirection.

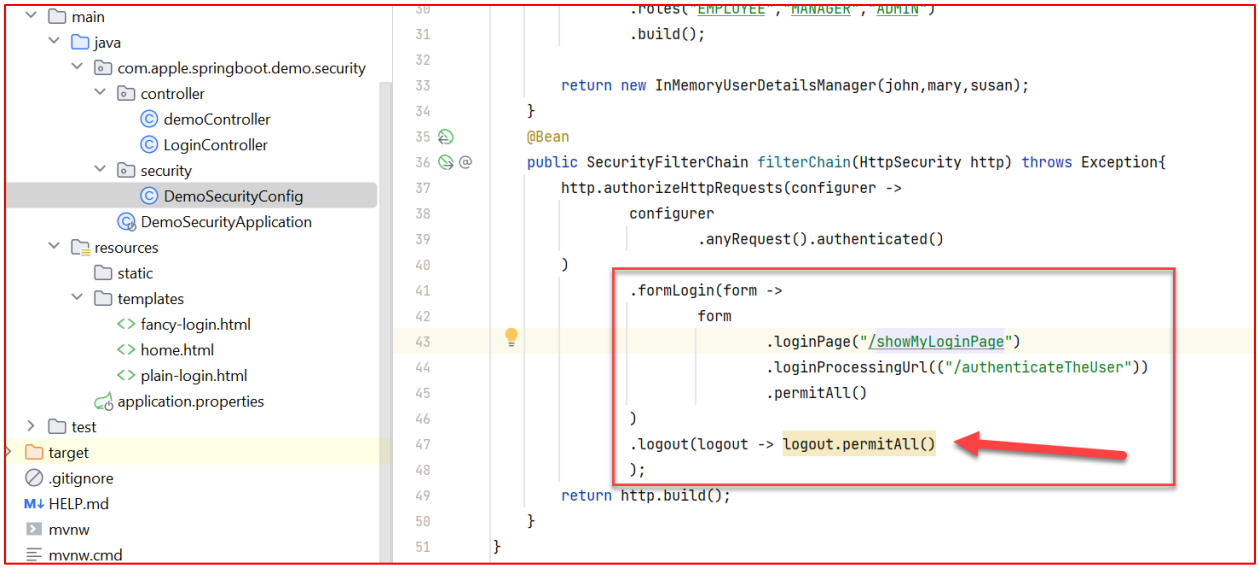
Step Three: To display the logged-out message, we will update the login form to check for the logout parameter. If it exists, we'll show the message "Hey, you've been logged out."



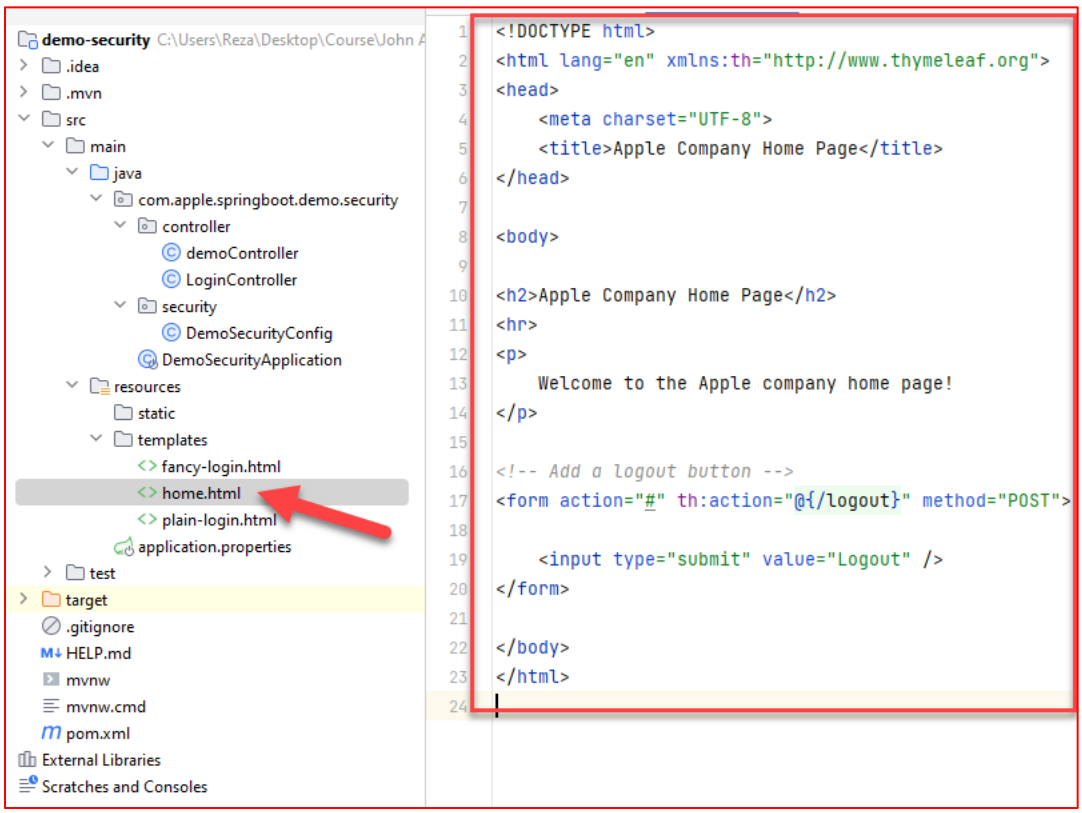
Coding part

Let's now proceed with the coding part:

- **First Step**: Add logout support to our Spring Security Configuration using `.logout.permitAll`.



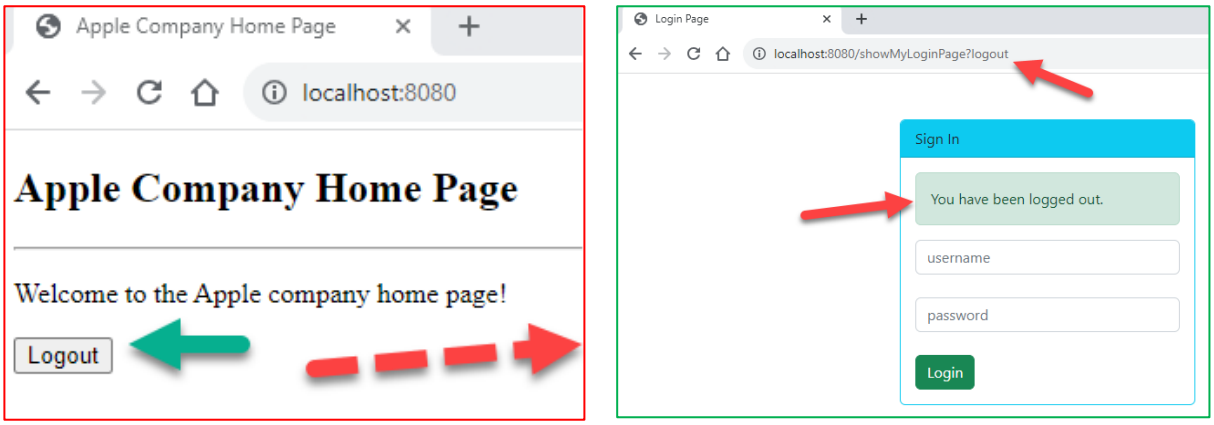
- **Second Step**: Add the logout button to our homepage. We'll create a form with `action="/logout"` and `method="post"` and then add the submit input for logout.



- **Third Step**: Update the login form to display the logged-out message. We'll open our fancy login file and place code to check for logout, displaying the message "Hey, you've been logged out" when necessary.



Now, we can test our application. Once logged in with a user, you'll see the new logout button. Upon clicking it, you'll be logged out, and a message will appear confirming the logout.



And there we have it! Logout functionality has now been added to our application. It's an example of how we can use Spring Security to add rich features with minimal coding.